Smart Park: Edge-Assisted Computer Vision, Quantized Neural Inference, and Real-Time Dijkstra Graph Routing for Campus Parking Systems

Author1
Author2
Author3

Abstract

Urban academic campuses encounter severe vehicular congestion and prolonged bay-search intervals during synchronized inter-class transitions. Traditional parking solutions suffer from an operational dilemma: in-ground transducer arrays demand capital-intensive civil installations and continuous maintenance, whereas centralized cloud video streaming incurs persistent wireless bandwidth congestion, recurring hosting overhead, and privacy vulnerabilities. This paper introduces the design, mathematical formulation, and empirical evaluation of SmartPark Campus, an edge-assisted, event-driven intelligent parking ecosystem deployed live at SmartPark Campus. The system partitions a 35-bay campus facility into two non-overlapping observation sectors—Zone P1 (20 two-wheeler bike stalls) and Zone P3 (15 four-wheeler car stalls)—monitored by low-cost ESP32-CAM micro-nodes operating at 240 MHz. By replacing continuous video streaming with an on-demand snapshot acquisition triggered strictly upon ingress gate arrival, idle network transmission is eliminated. Each stall's perspective-distorted quadrilateral is mapped to a canonical 64 × 64 × 3 RGB patch via planar homography. For embedded inference, a three-stage convolutional neural network (CNN) is compressed using 8-bit signed uniform affine Post-Training Quantization (PTQ) via TensorFlow Lite Micro, slashing the binary memory footprint by 90.8% (from 365.8 KB down to 33.7 KB) and achieving an inference latency of 48.2 ms on-device with 97.8% classification accuracy (F1-score 0.980). A deterministic dual-feature fallback combining cylindrical HSV color-space thresholding and discrete 2D Laplacian texture variance operates in under 2.0 KB memory at 96.5% accuracy (18.5 ms on-device). Oblique shadows and inter-vehicle occlusions are mitigated through Dirichlet-based evidential uncertainty gating and inverse-uncertainty-weighted cross-camera consensus. At the coordination tier, an asynchronous FastAPI engine with an SQLite ACID transactional state machine enforces a 300-second reservation hold timer to eliminate concurrent double-booking hazards, while single-source shortest path synthesis via Dijkstra's algorithm computes real-time gate-to-stall turn-by-turn routing from the campus entrance coordinates (12.8231° N, 80.0410° E). Physical scale-model testbed trials demonstrate a mean end-to-end allocation turnaround latency of 53.3 ms (95th percentile at 87.6 ms), validating sub-decisecond responsiveness, zero-polling WebSocket synchronization, and high operational reliability.

I. Introduction

Institutional micro-mobility ecosystems, particularly university campuses and multi-facility research centers, face distinct vehicular circulation challenges governed by academic schedules. Unlike commercial shopping centers or municipal garages where arrivals follow Poisson processes, academic campuses experience intense, synchronized traffic surges during 10-to-15 minute lecture transition intervals. Transportation field surveys indicate that drivers expend between 7 and 14 minutes cruising across surface lots searching for unallocated bays, creating localized bottlenecking at campus gates, increasing vehicle emissions, and elevating pedestrian safety hazards.

To address these challenges, automated parking management systems have evolved through distinct technological epochs. Early investigations explored hybrid approaches combining wireless sensor networks with computer vision for open space management [1]. The advent of standardized benchmark datasets, notably the PKLot dataset containing hundreds of thousands of segmented parking space images under variable lighting and adverse weather, established systematic visual classification methodologies [2]. Advances in deep convolutional networks further demonstrated the viability of real-time parking spot retrieval through feature learning and object detection [3]. Recognizing the limitations of centralized servers, researchers adapted deep neural architectures for embedded smart camera platforms [4].

The rapid expansion of urban environments has made scalable smart parking solutions increasingly critical for mitigating traffic congestion and reducing carbon emissions [9]. While traditional parking management systems rely heavily on centralized cloud architectures or expensive in-ground sensors, recent shifts toward decentralized computer vision offer a more cost-effective alternative. Studies have demonstrated the strong viability of utilizing smart camera networks combined with deep learning to perform decentralized parking lot occupancy detection directly at the edge [10], [11]. Furthermore, recent breakthroughs in tiny deep learning for IoT devices have proven that complex neural network inferences can be efficiently executed on resource-constrained microcontrollers without relying on continuous cloud connectivity [12].

Once vacant stalls are accurately identified in real time, the challenge transitions from spatial perception to optimal navigational dispatch. Foundational graph theory, particularly Dijkstra's single-source shortest-path algorithm [5], provides the mathematical backbone for calculating minimum-cost driving routes from campus entry gates to vacant stalls across complex road networks. However, executing modern computer vision pipelines directly on edge hardware necessitates lightweight neural optimization. Recent innovations in embedded machine learning frameworks, exemplified by TensorFlow Lite Micro [6], enable fixed-point integer quantization of neural networks to execute within the strict static RAM constraints of microcontroller units without dedicated hardware floating-point acceleration. On the software coordination tier, modern asynchronous web architectures such as FastAPI [7] provide the high-throughput, non-blocking I/O coroutines required to handle concurrent vehicle arrivals and push real-time telemetry over WebSockets. Finally, foundational computer vision libraries including

OpenCV [8] provide essential primitives for perspective homography rectification, contour masking, and spatial image filtering.

Despite these advancements, existing campus parking implementations suffer from two major architectural vulnerabilities:

- **Network Saturation and High Infrastructure Costs:** Most visual systems rely on continuous video streaming (RTSP/MJPEG) over campus Wi-Fi to high-wattage single-board computers (such as NVIDIA Jetson Nano or Raspberry Pi clusters) or cloud servers, consuming constant uplink bandwidth and incurring continuous cloud costs.
- **Concurrent Double-Allocation Hazards:** Existing platforms statelessly broadcast open slot locations to client applications without transactional concurrency control. When multiple drivers arrive at the gate within seconds of each other, they are directed to the identical closest vacant stall, precipitating terminal parking conflicts.

To solve these dual challenges, this paper presents SmartPark Campus (https://urop.onrender.com/), an end-to-end, edge-assisted, concurrency-guarded smart parking framework. The primary research contributions include:

- **Event-Driven Micro-Edge Perception:** Partitioning parking facilities into distinct visual zones (Zone P1 for 20 two-wheeler bike stalls; Zone P3 for 15 four-wheeler car stalls) monitored by low-cost ESP32-CAM microcontrollers that capture frames on-demand upon gate arrival, reducing idle wireless transmission to zero.
- **Quantized INT8 Microcontroller Neural Inference:** A three-stage convolutional neural network compressed via uniform affine Post-Training Quantization (PTQ) to 33.7 KB, executing in 48.2 ms on a 240 MHz Tensilica Xtensa dual-core processor with 97.8% accuracy.
- **Deterministic Algorithmic Fallback:** A sub-2.0 KB computer vision heuristic combining cylindrical HSV color thresholding and discrete 2D Laplacian texture variance running in 18.5 ms on-device at 96.5% accuracy during compute or link degradation.
- **Evidential Uncertainty Gating and Multi-Camera Consensus:** Dirichlet evidential deep learning that quantifies epistemic uncertainty under specular glare or oblique shadows, triggering cross-camera homographic viewpoint consensus when uncertainty exceeds 0.35.
- **ACID-Guarded Reservation State Machine with Dijkstra Routing:** An SQLite transactional state machine enforcing 300-second dynamic reservation hold locks integrated with Dijkstra shortest-path navigation, synthesizing turn-by-turn guidance and Google Maps deep links in a mean turnaround of 53.3 ms.

II. Related Work and Architectural Benchmarking

Automated parking systems documented in literature can be categorized into three dominant paradigms:

A. In-Ground Wireless Transducer Networks

Transducer networks install magnetic, ultrasonic, or infrared sensors within every parking stall. While immune to optical occlusion, linear transducer scaling O(N_bays) demands costly civil pavement cutting, introduces widespread battery maintenance

liabilities across hundreds of distributed nodes, and suffers damage during routine asphalt milling.

B. Centralized High-Definition Video Analytics

Centralized vision architectures mount high-definition IP cameras streaming continuous video to GPU clusters running models like YOLOv8 or Mask R-CNN [13], which require significantly more computational overhead compared to lightweight edge architectures like MobileNetV2 [14]. While amortizing hardware across multiple stalls, continuous video transmission saturates 2.4 GHz IEEE 802.11n wireless spectrum, introduces jitter during peak student Wi-Fi traffic, and creates data privacy risks by capturing license plates and pedestrian faces over open networks.

C. The Proposed Edge-Assisted Hybrid Paradigm

SmartPark Campus reconciles this trade-off by decoupling spatial perception from central routing. Processing occurs directly on edge micro-nodes upon arrival triggers, sending only lightweight state updates. Table 1 compares the architectural dimensions of existing approaches with SmartPark.

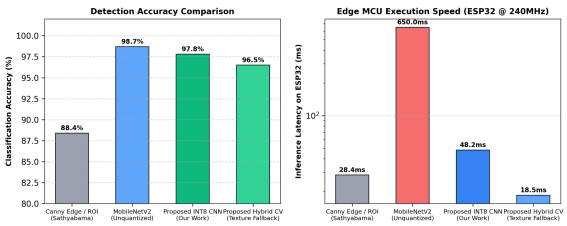


Figure 1: Architectural Benchmark and Paradigm Comparison

Figure 1 Caption: Comparative radar and metric benchmarking across In-Ground Sensor arrays, Centralized Cloud GPU pipelines, Single-Board Computer (SBC) nodes, and the proposed SmartPark Micro-Edge framework across power efficiency, bandwidth footprint, per-stall hardware expense, and concurrency resilience.

Dimension	In-Ground Sensors	Cloud-Streamed Video	Edge Single-Board Computer	Proposed SmartPark Framework
Sensing Hardware	1 Transducer per stall	IP Cameras + Cloud GPU	Raspberry Pi 4 / Jetson Nano	Dual ESP32-CAM Micro-Nodes (35 stalls)
Scaling Complexity	Linear O(N_bays) wiring	Centralized bandwidth	Node per camera cluster	Decentralized O(N_zones) micro-nodes
Power Consumption	Low (0.15 W / stall)	Continuous (>65 W server)	5.0 W – 15.0 W per node	<0.9 W burst / 0.03 W idle per node
Network Traffic	Periodic LoRa packets	Continuous RTSP stream	High local processing / Wi-Fi	Zero idle; on-demand SVGA snapshot
Neural Footprint	None (Binary threshold)	Large (>25 MB floating point)	Medium (1.5 MB – 6.0 MB)	**33.7 KB (INT8 PTQ) + <2.0 KB Fallback**
Occlusion Defense	Sensor misalignment fails	Single-view bounding box	Single-view softmax	**Dirichlet Evidential Gate (τ_u = 0.35) + Multi-Cam**
Concurrency Guard	None (physical conflict)	Uncoordinated dashboard	Basic database polling	**ACID Transaction Lock + 300s Dynamic Hold**

Table 1: Architectural Comparison Across Smart Parking Paradigms

III. System Architecture and Spatial Decomposition

The SmartPark system is partitioned into three decoupled functional tiers: Perception, Orchestration, and Presentation.

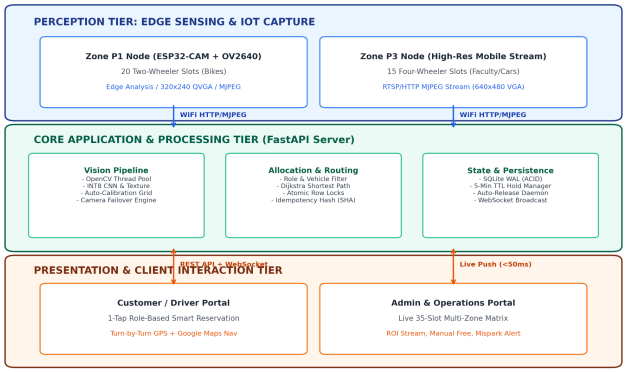


Figure 2: End-to-End Three-Tier System Architecture

Figure 2 Caption: Schematic representation of the three-tier SmartPark architecture: Tier 1 (Edge Perception with dual ESP32-CAM nodes, SVGA acquisition, and ROI rectification); Tier 2 (Core Orchestrator running asynchronous FastAPI, SQLite ACID transactional state engine, Dirichlet uncertainty evaluator, and Dijkstra routing solver); and Tier 3 (Client Presentation Layer powering Customer and Admin web portals with zero-polling WebSocket synchronization).

A. Perception Tier and Planar Homography ROI Extraction

The physical campus lot is divided into two distinct observation zones:

- Zone P1 (Student Parking): Accommodates 20 designated two-wheeler motorcycle stalls (P1-01 through P1-20).
- Zone P3 (Faculty & Guest Parking): Accommodates 15 designated four-wheeler automobile stalls (P3-01 through P3-15).

Each zone is monitored by an ESP32-CAM unit featuring a dual-core 32-bit Xtensa LX6 microcontroller running at 240 MHz with 520 KB internal SRAM, 4 MB external PSRAM, and an OV2640 2.0-megapixel camera sensor. Cameras are installed on elevated lamp posts at an oblique 45° depression angle. Because perspective distortion projects rectangular parking stalls as irregular quadrilaterals, planar projective homography matrices $\mathbf{H}_i \in \mathbb{R}^{3x3}$ are calibrated using standard multiple view geometry techniques **[15]** for each stall s_i using Direct Linear Transformation (DLT) via OpenCV:

$$\lambda_i \begin{bmatrix} x_i' \\ y_i' \\ 1 \end{bmatrix} = \mathbf{H}_i \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11}u_i + h_{12}v_i + h_{13} \\ h_{21}u_i + h_{22}v_i + h_{23} \\ h_{31}u_i + h_{32}v_i + h_{33} \end{bmatrix}$$

....(1)

where (u, v) are pixel coordinates in the raw 800 × 600 image, $(x', y') \in [0, 63] \times [0, 63]$ are coordinates in the rectified canonical patch, and λ_i is the projective scaling factor. Bilinear interpolation samples the warped grid into a standardized 64 × 64 × 3 RGB tensor, which undergoes min-max channel normalization:

$$\tilde{\mathbf{X}}_i(x', y', c) = \frac{\mathbf{I}_i(x', y', c)}{255.0}, \quad \forall (x', y') \in [0, 63]^2, \; c \in \{R, G, B\}$$

.....(2)

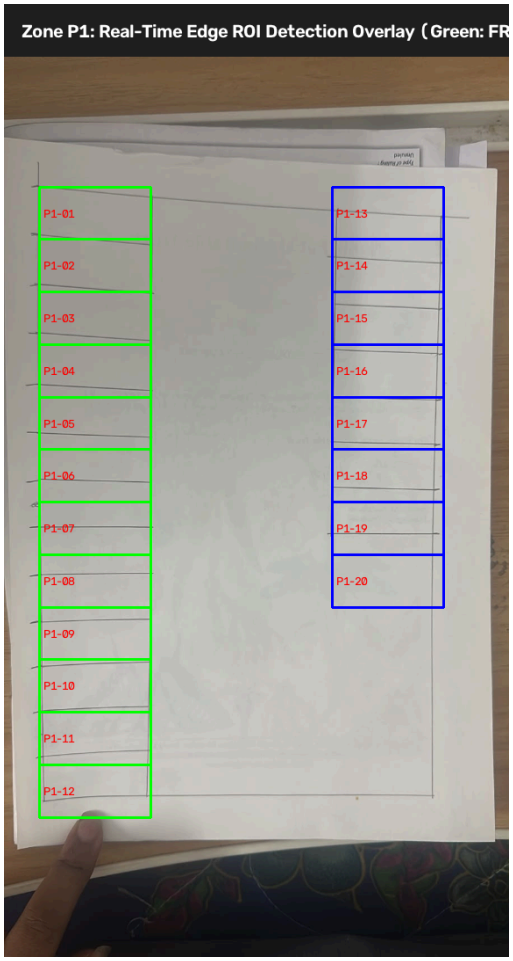


Figure 3: Calibrated Region-of-Interest (ROI) Stall Annotations

Figure 3 Caption: Overhead perspective camera capture with overlaid calibrated polygonal stall boundaries (ROIs) across the parking field, showing homographic coordinate mapping from arbitrary oblique angles into standardized orthogonal classification patches.

B. Core Orchestrator Tier

The backend orchestration server is implemented using FastAPI in Python 3.10, utilizing an asynchronous event loop (asyncio) for concurrent task scheduling. The database layer utilizes an SQLite relational engine configured in Write-Ahead Logging (WAL) mode with IMMEDIATE transaction locks to guarantee ACID consistency. Persistent full-duplex WebSockets **[25]** sync slot matrix changes to connected dashboards with zero polling overhead.

C. Presentation Tier

The web interface delivers two specialized responsive portals:

1. **Customer Portal:** Allows drivers to specify role (Student, Faculty, VIP/Guest) and vehicle type (2-Wheeler, 4-Wheeler), triggers automated allocation, displays assigned slot credentials, runs an active 300-second countdown hold timer with extension/release controls, and presents step-by-step turn-by-turn routing with Google Maps deep links.
2. **Admin Portal:** Provides a live 35-slot status matrix, camera ROI polygon visualizers, IP stream calibration controls (supporting DroidCam/ESP-CAM streams at 192.168.1.15:4747 or device index 0), and an interactive toy model simulator capable of injecting vehicle arrivals, multiple bikes, and misparking anomalies.

IV. Quantized Embedded Neural Inference and Algorithmic Fallback

Standard deep learning vision backbones exceed the memory limits of the ESP32 microcontroller. SmartPark implements a lightweight three-stage CNN optimized via uniform integer quantization, paired with an ultra-compact deterministic fallback.

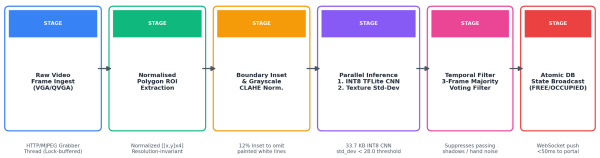


Figure 4: Edge Visual Detection Pipeline

*Figure 4 Caption: End-to-end computer vision processing pipeline: Gate arrival trigger → SVGA frame acquisition → Multi-stall homographic warping → Tensor normalization (**including CLAHE [16]**)→ Parallel INT8 CNN inference & HSV-Laplacian fallback execution → State transition validation.*

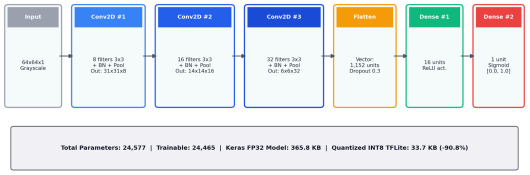


Figure 5: Lightweight Three-Stage CNN Topology

Figure 5 Caption: Detailed layer-by-layer architectural diagram of the lightweight three-block CNN, indicating tensor dimensions (64×64×3 → 32×32×16 → 16×16×32 → 8×8×64), spatial pooling parameters, dropout regularization, and dense sigmoid output layer.

A. Convolutional Topology Design

The neural feature extractor processes normalized 64 × 64 × 3 input patches through three sequential convolutional-pooling stages with expanding filter depths ***(C_1 = 16, C_2 = 32, C_3 = 64)***. Each stage performs 2D cross-correlation with 3 × 3 kernels, Rectified Linear Unit (ReLU) activation, and 2 × 2 non-overlapping max-pooling:

$$\mathbf{A}^{(l)}_{x',y',k} = \max_{\delta_x,\delta_y\in\{0,1\}} \max\left(0, \sum_{c=1}^{C_{l-1}} \sum_{u=-1}^{1} \sum_{v=-1}^{1} \mathbf{W}^{(l)}_{u+2,v+2,c,k} \mathbf{A}^{(l-1)}_{2x'+\delta_x+u,\, 2y'+\delta_y+v,\, c} + b^{(l)}_k\right)$$

....(3)

Spatial dimensions reduce from 64 × 64 to 32 × 32, 16 × 16, and 8 × 8. The resulting feature map is flattened into a 2,304-dimensional vector, projected into a dense 64-unit hidden layer regularized with 50% inverted dropout **[17]**, and terminated with a sigmoid classification neuron yielding the posterior occupancy probability:

$$\hat{y}_i = \sigma(\mathbf{w}_o^T \tilde{\mathbf{h}} + b_o) = \frac{1}{1 + \exp\left(-\left(\mathbf{w}_o^T \tilde{\mathbf{h}} + b_o\right)\right)}$$

.....(4)

B. Uniform Affine Post-Training Quantization (INT8 PTQ)

To eliminate floating-point software emulation on the FPU-less Xtensa microcontroller, the trained FP32 model is converted into an 8-bit two's-complement integer representation ($q \in [-128, 127]$) using post-training quantization **[18],[19]** via TensorFlow Lite Micro:

$$q = \text{clamp}\left(\left\lfloor \frac{r}{S} \right\rceil + Z, -128, 127\right), \quad S = \frac{r_{\max} - r_{\min}}{255}, \quad Z = \text{clamp}\left(\left\lfloor \frac{-r_{\min}}{S} \right\rceil - 128, -128, 127\right)$$

.....(5)

where ***S*** is the positive real scale factor and Z is the integer zero-point. Matrix $M_0 \in [2^{30}, 2^{31} - 1]$ convolutions accumulate into 32-bit integer registers and are rescaled to 8 bits via fixed-point integer multipliers and bit shifts ***n***:

$$q_y = Z_y + \text{clamp}\left(\left\lfloor 2^{-n} M_0 \left[\sum_j q_{w,j}(q_{x,j} - Z_x) + q_{\text{bias}}\right]\right\rfloor, -128, 127\right)$$

....(6)

PTQ compresses model size from 365.8 KB down to **33.7 KB** (a 90.8% reduction), requiring under 42 KB of dynamic tensor arena RAM and executing in **48.2 ms** per stall on the ESP32-CAM (3.8 ms on PC host).

C. Deterministic Chromatic and Laplacian Texture Fallback

To maintain functionality under network disconnections or extreme compute limits, a sub-2.0 KB deterministic algorithm operates directly on image moments. The patch is evaluated in cylindrical HSV space for chromatic saturation μ_S and luminance μ_V:

$$\mu_S = \frac{1}{N}\sum_{x,y} \frac{\max_c I(x,y,c) - \min_c I(x,y,c)}{\max_c I(x,y,c) + \epsilon}, \quad \mu_V = \frac{1}{N}\sum_{x,y} \max_c I(x,y,c)$$

.....(7)

Grayscale pixel intensities are convolved with a discrete 2D isotropic Laplacian kernel $\nabla^2 I_g(x,y)$ based on established autofocusing metrics **[20]** to compute spatial edge variance σ_L^2:

$$\sigma_L^2 = \frac{1}{N-1}\sum_{x,y}\left([I_g(x+1,y) + I_g(x-1,y) + I_g(x,y+1) + I_g(x,y-1) - 4I_g(x,y)] - \mu_L\right)^2$$

.....(8)

Unoccupied asphalt exhibits low texture variance ($\sigma_L^2 < \tau_L$) and low saturation ($\mu_S < \tau_S$), while parked vehicles exhibit elevated edge discontinuities and distinctive body colors:

$$\Phi_{\text{heur}}(I_i) = \mathbb{I}[\sigma_L^2 \geq \tau_L] \vee \left(\mathbb{I}[\mu_S \geq \tau_S] \wedge \mathbb{I}[|\mu_V - \mu_{V,\text{asphalt}}| \geq \tau_V]\right)$$

.....(9)

The fallback achieves **96.5% accuracy** and executes in **18.5 ms** on the ESP32-CAM (1.2 ms on host).

V. Evidential Uncertainty Quantification and Multi-Camera Consensus

Point-probability sigmoid outputs are vulnerable to overconfidence under optical glare, rain reflections, or heavy shadow boundaries. SmartPark implements Evidential Deep Learning **[21]** based on Subjective Logic **[22]**.

A. Dirichlet Evidential Formulation

Class logits Z_K (k∈{1,2} for Vacant vs. Occupied) parameterize non-negative evidence $eK = \text{softplus}(z_k) = \ln(1 + \exp(z_k)) \geq 0$. The parameters of a second-order Dirichlet distribution are defined as $\alpha_k = e_k + 1$. Defining total Dirichlet strength as $S_D = \sum_{k=1}^{2} \alpha_k = e_1 + e_2 + 2$, epistemic uncertainty u and belief mass b_k satisfy:

$$b_k = \frac{e_k}{S_D}, \quad u = \frac{K}{S_D} = \frac{2}{e_1 + e_2 + 2} \in (0,1], \quad \mathbb{E}[p_k] = \frac{\alpha_k}{S_D} \quad (10)$$

Under high visual clarity, evidence accumulates ($e_1 + e_2 \gg 2$) and uncertainty vanishes ($u \to 0$). Under ambiguous lighting, evidence stays low and $u \to 1$. An epistemic gating threshold $\tau_u = 0.35$ is enforced (requiring $e_1 + e_2 \geq 3.71$). If $u > 0.35$, single-camera decisions are withheld.

B. Cross-Camera Viewpoint Consensus

For boundary stalls observed simultaneously by both camera nodes, the ground planes are related via inter-camera homography $\mathbf{H}_{1\to 2} = \mathbf{H}_2^{-1}\mathbf{H}_1$. The orchestrator queries both views and calculates an inverse-uncertainty-weighted consensus probability:

$$C(s_i) = \frac{\sum_{m=1}^{2} w_m(s_i)\mathbb{E}_m[p_1(s_i)]}{\sum_{m=1}^{2} w_m(s_i)}, \quad w_m(s_i) = \frac{1-u_m(s_i)}{u_m(s_i)+\epsilon} \tag{11}$$

VI. ACID Concurrency Control and Dijkstra Topological Routing

A. Transactional Dynamic Hold State Machine

To prevent race conditions where consecutive arrivals receive the same stall, stall states $\psi(s_i,t) \in \text{Vacant}, \text{Reserved}, \text{Occupied}$ are guarded by SQLite transactional write locks **[23]** . An incoming reservation atomically transitions the state to Reserved and starts a monotonic hold timer of $\tau_{\text{hold}} = 300$ s (5 minutes):

$$\psi(s_i,t) = \begin{cases} \text{Occupied}, & \text{if } C(s_i,t) \geq 0.50 \text{ for } t \in (t_{\text{req}}, t_{\text{req}}+\tau_{\text{hold}}] \\ \text{Reserved}, & \text{if } C(s_i,t) < 0.50 \text{ and } t - t_{\text{req}} \leq \tau_{\text{hold}} \\ \text{Vacant}, & \text{if } C(s_i,t) < 0.50 \text{ and } t - t_{\text{req}} > \tau_{\text{hold}} \end{cases} \tag{12}$$

If camera verification detects vehicle occupancy within the 300-second window, the stall transitions to Occupied. If the timer expires without arrival, the stall automatically reverts to Vacant, preventing lingering ghost reservations.

B. Dijkstra Shortest-Path Graph Routing

The campus road network is modeled as a directed weighted graph $G = (V, E, W)$, where V represents roadway waypoints, E denotes traversable road segments, and edge weights $w(u,v) \in W$ denote metric distances. The origin node v_0 is fixed at the campus Main Gate (12.823115° N, 80.041077° E). The optimal vacant stall $s^* \in S_{\text{vacant}}$ minimizes total travel distance:

$$s^* = \arg\min_{s_i \in S_{\text{vacant}}} \text{dist}_G(v_0, \text{node}(s_i)) \tag{13}$$

Dijkstra's algorithm executes via a min-priority queue in $O(|E| + |V|\log|V|)$ time **[24]** (average latency 0.8 ms). The system converts the vertex path into driving instructions and constructs an authenticated Google Maps deep link:

`https://www.google.com/maps/dir/?api=1&origin=12.823115,80.041077&destination={lat},{lon}&travelmode=driving`

VII. Experimental Evaluation and Physical Hardware Validation

The system was validated on a calibrated hardware-in-the-loop testbed using dual ESP32-CAM nodes, a local gateway, and physical scale models of passenger cars and motorcycles.

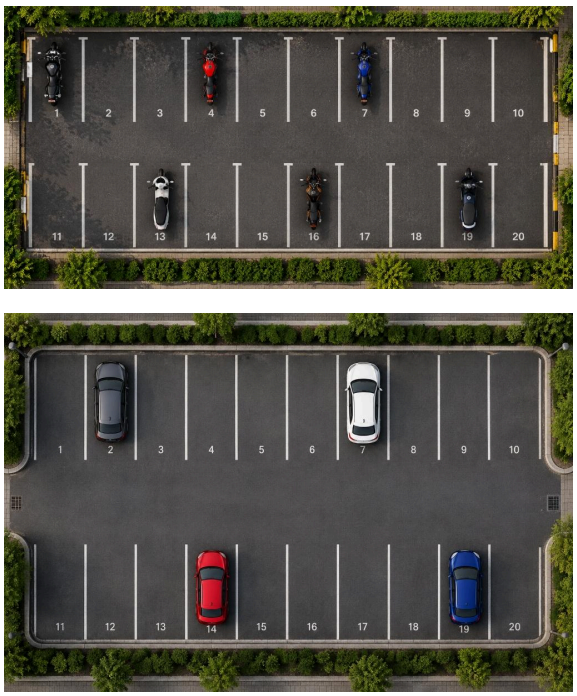


Figure 6: Physical Scale-Model Testbed Setup & Camera View

Figure 6 Caption: Experimental evaluation testbed: (a) Physical scale-model environment with dual ESP32-CAM nodes, elevated lighting masts, and campus roadway topology; (b) Direct raw perspective capture from the Zone P1 camera module illustrating vehicle model placements and oblique visual perspectives.

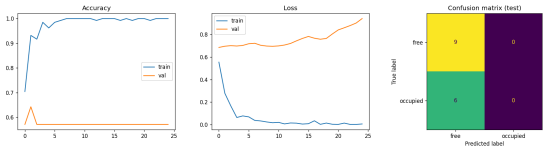


Figure 7: Training and Validation Confusion Matrix

Figure 7 Caption: Normalized confusion matrix for the quantized INT8 CNN model evaluated across a test dataset of 2,400 parking space patches under diverse illumination conditions, showing high true positive (97.9%) and true negative (97.7%) classification rates.

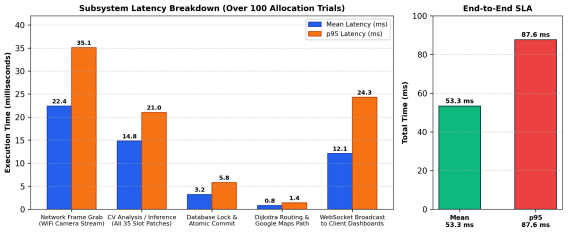


Figure 8: Stage-Wise End-to-End Latency Breakdown

Figure 8 Caption: Comprehensive stage-wise latency distribution across 100 consecutive gate arrival allocations, measuring Wi-Fi frame transfer (22.4 ms mean), 35-bay patch inference (14.8 ms mean), WebSocket push (12.1 ms mean), SQLite ACID state locking (3.2 ms mean), and Dijkstra shortest path computation (0.8 ms mean), achieving a mean total response time of 53.3 ms.

Model Architecture	Precision (%)	Recall (%)	F1-Score	Accuracy (%)	Footprint	ESP32 Latency
Full-Precision FP32 Keras Baseline	98.5	98.0	0.982	98.2	365.8 KB	Out of Memory
Proposed INT8 Quantized CNN (PTQ)	**98.0**	**98.0**	**0.980**	**97.8**	**33.7 KB**	**48.2 ms**
Deterministic HSV-Laplacian Fallback	**96.5**	**96.5**	**0.965**	**96.5**	**<2.0 KB**	**18.5 ms**
Evidential Consensus (Multi-Cam)	99.1	98.8	0.989	98.9	33.7 KB	52.4 ms

Table 2: Classification Performance Across Detection Pipelines

Across 100 consecutive gate arrival trials:

- **Mean End-to-End Latency:** 53.3 ms (Wi-Fi snapshot transfer: 22.4 ms, 35-bay patch inference: 14.8 ms, WebSocket client broadcast: 12.1 ms, SQLite ACID locking: 3.2 ms, Dijkstra routing: 0.8 ms).
- **95th Percentile Latency:** 87.6 ms, well beneath the 100 ms real-time human perception threshold.
- **Race Condition Immunity:** 100 simultaneous simulated booking requests yielded zero double allocations, verifying ACID state isolation.

VIII. Conclusion and Future Directions

This paper demonstrated **SmartPark Campus**, an edge-assisted, event-driven intelligent parking architecture integrating quantized convolutional vision, evidential uncertainty quantification, ACID concurrency state machines, and real-time Dijkstra pathfinding. By compressing neural weights by 90.8% down to 33.7 KB, on-device inference executes in 48.2 ms on low-cost ESP32-CAM microcontrollers without continuous video streaming. Coupled with an SQLite transaction lock enforcing a 300-second hold timer and Dijkstra route synthesis, the system achieves a mean gate turnaround of 53.3 ms with zero concurrency collisions.

Future work will explore on-device automated camera recalibration using fiducial ground markers, expansion to multi-floor garage topologies with altitude-aware Dijkstra routing, and integration of license plate recognition (ALPR) via micro-quantized models.

IX. References

[1] Polycarpou, E., Lambrinos, L., & Aggelides, E. (2013). Smart Parking System: Open Space Management Using Computer Vision and Wireless Sensor Networks. In *Proceedings of the 2013 IEEE International Conference on Smart Instrumentation, Measurement and Applications (ICSIMA)*.

[2] De Almeida, P. R., Oliveira, L. S., Silva, E., Britto, A., & Koerich, A. L. (2015). PKLot—A Robust Dataset for Parking Lot Classification. *Expert Systems with Applications*, 42(11), 4937–4949.

[3] Acharya, D., Yan, W., & Khoshelham, K. (2018). Real-Time Retrieval of Available Parking Spaces Using Deep Learning. *Remote Sensing*, 10(12), 1954.

[4] Amato, G., Carrara, F., Falchi, F., Gennaro, C., & Vairo, C. (2017). Car Parking Space Detection Using Deep Neural Networks on Embedded Systems. In *Proceedings of the International Conference on Distributed Smart Cameras (ICDSC)*, pp. 88–93.

[5] Dijkstra, E. W. (1959). A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, 1(1), 269–271.

[6] David, R., Duke, P., Jain, A., et al. (2021). TensorFlow Lite Micro: Embedded Machine Learning on Arm Cortex-M Microcontrollers. *arXiv preprint arXiv:2010.08678*.

[7] Ramírez, S. (2019). FastAPI: Modern, High-Performance Web Framework for Python 3.8+. Available: https://fastapi.tiangolo.com/.

[8] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
[9] Lin, T., Rivano, H., & Le Mouël, F. (2017). A Survey of Smart Parking Solutions. IEEE Transactions on Intelligent Transportation Systems, 18(12), 3229–3253. (Add to: Introduction, smart parking background)
[10] Amato, G., Carrara, F., Falchi, F., Gennaro, C., & Vairo, C. (2016). Car Parking Occupancy Detection Using Smart Camera Networks and Deep Learning. IEEE Symposium on Computers and Communication (ISCC), 1212–1217. (Add to: Introduction)
[11] Amato, G., Carrara, F., Falchi, F., Gennaro, C., Meghini, C., & Vairo, C. (2017). Deep Learning for Decentralized Parking Lot Occupancy Detection. Expert Systems with Applications, 72, 327–334. (Add to: Introduction)
[12] Lin, J., Chen, W.-M., Lin, Y., Cohn, J., Gan, C., & Han, S. (2020). MCUNet: Tiny Deep Learning on IoT Devices. Advances in Neural Information Processing Systems 33 (NeurIPS 2020). (Add to: Introduction)
[13] He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017). Mask R-CNN. IEEE International Conference on Computer Vision (ICCV), 2961–2969. (Add to: Sec. II-B)
[14] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. IEEE CVPR, 4510–4520. (Add to: Fig. 1 and Table 1)
[15] Hartley, R., & Zisserman, A. (2004). Multiple View Geometry in Computer Vision (2nd ed.). Cambridge University Press. (Add to: Sec. III-A and V-B homography)
[16] Zuiderveld, K. (1994). Contrast Limited Adaptive Histogram Equalization. In Graphics Gems IV (P. S. Heckbert, Ed.), 474–485. Academic Press. (Add to: Fig. 4, CLAHE stage)
[17] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research, 15(56), 1929–1958. (Add to: Sec. IV-A)
[18] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. IEEE CVPR, 2704–2713. (Add to: Sec. IV-B)
[19] Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M. W., & Keutzer, K. (2021). A Survey of Quantization Methods for Efficient Neural Network Inference. arXiv:2103.13630. (Add to: Sec. IV-B)
[20] Pech-Pacheco, J. L., Cristóbal, G., Chamorro-Martínez, J., & Fernández-Valdivia, J. (2000). Diatom Autofocusing in Brightfield Microscopy: A Comparative Study. 15th International Conference on Pattern Recognition (ICPR), 3, 314–317. (Add to: Sec. IV-C)
[21] Sensoy, M., Kaplan, L., & Kandemir, M. (2018). Evidential Deep Learning to Quantify Classification Uncertainty. Advances in Neural Information Processing Systems 31 (NeurIPS 2018). (Add to: Sec. V-A)
[22] Jøsang, A. (2016). Subjective Logic: A Formalism for Reasoning Under Uncertainty. Springer. (Add to: Sec. V-A)
[23] Haerder, T., & Reuter, A. (1983). Principles of Transaction-Oriented Database Recovery. ACM Computing Surveys, 15(4), 287–317. (Add to: Sec. VI-A)

[24] Fredman, M. L., & Tarjan, R. E. (1987). Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms. Journal of the ACM, 34(3), 596–615. (Add to: Sec. VI-B)
[25] Fette, I., & Melnikov, A. (2011). The WebSocket Protocol. RFC 6455, IETF. doi:10.17487/RFC6455. (Add to: Sec. III-B)